Task 1C - Instructions

Learn this first

Before you start, read Lane Detection **in the Learnings section.** It covers what you need for this task.

Aim

> Fill in the `detect_lane()` function of the Task 1C boilerplate so that, for every frame of the given videos, it reports **where the centre of the lane is** and **which lane the vehicle is in**.

Files and Setup

Both files come from the theme's public repository - see Getting the Code on the overview page.

File	What it is
`task1c/lane_detection.py`	the boilerplate you fill in
`task1c/public/`	the 20 video clips, already in the repository

You do not download or unzip anything: cloning the repository gives you the clips. The folder looks like this, and the evaluation tool finds them on its own:

```
task1c/
├── lane_detection.py      <- the boilerplate, with your detect_lane()
└── public/
    ├── clip_01.mp4        <- 20 clips, 640 x 480, 20 fps, 200 frames each
    ├── clip_02.mp4
    └── …
```

Then work inside your `NV_<Team-ID>` environment from Task 0:

```
conda activate NV_<Team-ID>
python lane_detection.py public/clip_01.mp4 --show
```

These clips are for developing on your own machine. Your submission is **also** run against a private set of clips you never see, so a detector tuned to these particular frames will not carry over. How the marks are split, and how close your `center_x` has to be, are on the Submission page - worth reading before you start optimising.

The Function You Implement

`detect_lane()` is the only function you write. It receives one frame and returns one dictionary:

```
def detect_lane(frame):
    ...
    return {"center_x": center_x, "lane": lane}
```

Key	Type	Meaning
`center_x`	`int`	x-pixel of the lane centre in this frame, or `-1` if the lane could not be found
`lane`	`str`	`"left"`, `"right"` or `"unknown"`

Which lane is which. The dashed white line divides the two lanes, so the side of it that the vehicle is on decides the answer:

The dashed centre line is...	`lane`
to the **right** of the vehicle	`"left"`
to the **left** of the vehicle	`"right"`
not identifiable in this frame	`"unknown"`

Return `"unknown"` and `-1` honestly when a frame defeats your detector. A wrong answer and a missing answer score the same, and a detector that reports failure is far easier to debug than one that guesses.

Frame Resolution and Coordinates

The clips are **640 × 480**, and `center_x` is an absolute pixel column in the frame **as you received it**. The evaluator compares it against a ground truth measured in those same pixels, so the number only means anything in them.

> ⚠ Report the answer in the frame's own coordinates
>
> Inside `detect_lane()` you may resize, crop or warp as much as you like - but convert the answer back before you return it.
>
> - Found the centre in a **320 × 240** copy? It is half the value it should be. Scale it back up.
> - Read it off a **bird's eye view**? That is a warped coordinate, not a frame one. Map the point back through the inverse of your perspective transform (`cv2.perspectiveTransform` with the inverse matrix).
> - Do **not** re-encode, resize or re-compress the clip files themselves. The evaluator checks every clip is 640 × 480 and refuses to grade a dataset that is not, because every correct answer would be shifted sideways.

Keeping one fixed resolution is what makes the same tolerance fair for everyone, whatever method you chose.

Adding Helper Functions

You are not expected to fit everything into one function. Add as many helper

functions and global variables as you like - put them **below** `detect_lane()` **but above the** `END OF YOUR IMPLEMENTATION` **line**, and call them from `detect_lane()`:

```
def detect_lane(frame):
    ...
    yellow_x, white_x = marking_positions(frame)      # your helper,
defined below
    ...
    return {"center_x": center_x, "lane": lane}


# ---- helpers, still inside the implementation block ----

def marking_positions(frame):
    ...
    return yellow_x, white_x
```

Defining a helper after `detect_lane()` is fine - Python resolves the name when the call actually happens, not when the file is read. What matters is that the helper sits inside the implementation block and that `detect_lane()` reaches it: the evaluation script only ever calls that one function, so anything it does not reach is dead code.

List any helpers and globals you add in the file header at the top of the script.

Rules

⚠ Read this before you start writing code

- Write your code **only** inside the block marked `ADD YOUR IMPLEMENTATION HERE`. Everything outside it - `validate_result()`, `draw_overlay()`, `process_video()`, `main()` - must stay exactly as shipped, because the evaluation script depends on it.
- **No display or printing inside** `detect_lane()`. No `cv2.imshow()`, `cv2.waitKey()`, `cv2.imwrite()` or `print()`. The function computes and returns, nothing else. All visualisation belongs in `draw_overlay()`, which `process_video()` already calls for you when you pass `--show`.
- Do not rename the file, the function, or the keys of the returned dictionary.
- Helper functions and global variables are welcome, but they must live inside the marked block and be reachable from `detect_lane()`.

Running It

```
# every clip in the folder, with the overlay window - press q to stop
python lane_detection.py public --show

# write the per-frame results out so you can inspect them
python lane_detection.py public --out results.json
```

Each clip prints a one-line summary of how many frames came back `left`, `right`, `unknown`, and how many had no centre at all:

```
clip_01.mp4                    200 frames | left   200 | right     0 |
unknown     0 | no centre     0
```

If `detect_lane()` returns something of the wrong shape, `validate_result()` stops the run and names the frame and the problem:

```
ValueError: detect_lane() on frame 12 returned lane = 'LEFTish',
expected one of left, right, unknown
```

Before You Submit

- Check the `--show` overlay through a **turn**, not only on the straights.
- Confirm the lane label stays stable when the dashed line is passing through a gap.
- Comment out or remove any debugging you added inside `detect_lane()`.